SAICC XVI · HANDS-ON

Checkpoints do app de tarefas

Seis etapas, cada uma com o código esperado e uma ilustração do resultado na tela

Matheus Silveira

Como funcionam os checkpoints

1. Acompanhe

Digite o código junto com o instrutor e use o hot reload (`r`) a cada passo.

2. Compare

Cada etapa termina em algo visível na tela. Veja a ilustração: o seu app está igual?

3. Restaure

Travou? Copie a pasta do checkpoint para o seu `lib/` e siga com a turma.

```
# na raiz do repositório clonado (exemplo: checkpoint 3)
cp -r checkpoints/03-lista-fixa/* /caminho/do/seu/todo_app/lib/
```

Cada pasta tem **exatamente** o que o `lib/` deve conter ao final da etapa. Os checkpoints são cumulativos: o seguinte só adiciona ou altera arquivos.

VISÃO GERAL

Os 6 checkpoints



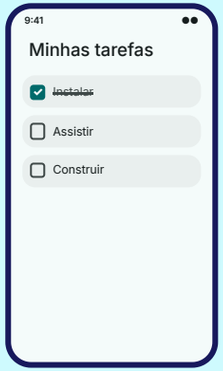
1

Projeto base



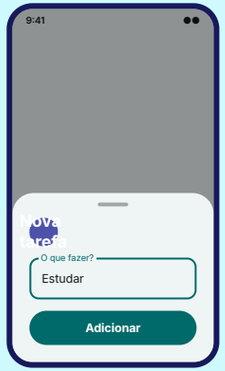
2

Tema



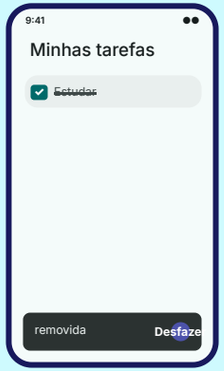
3

Lista fixa



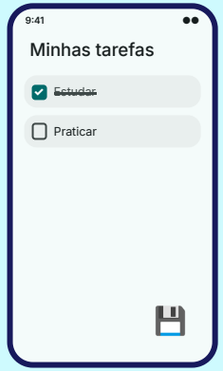
4

Adicionar



5

Concluir e remover



6

Persistência

Cada etapa acrescenta uma funcionalidade ao app: do "Olá" ao app que salva os dados.

CHECKPOINT 1

Projeto base

Pasta: `checkpoints/01-projeto-base`

`main.dart` NOVO

- Criar o projeto com `flutter create --empty`
- `runApp` recebe o widget raiz
- `MaterialApp` + `Scaffold`: o esqueleto de uma tela
- Hot reload: troque o texto e salve



Esperado: o texto no centro da tela

CHECKPOINT 1

Código: `main.dart`

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const TodoApp());
}

class TodoApp extends StatelessWidget {
  const TodoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'Minhas tarefas',
      home: Scaffold(
        body: Center(child: Text('Olá, SAICC!')),
      ),
    );
  }
}
```

```
flutter create --empty \
  --org br.edu.furg.saicc todo_app
cd todo_app
flutter run
```

Para comentar

- Tudo na tela é um **widget**
- `const` evita reconstruções inúteis
- `Scaffold`: app bar, corpo e botão flutuante

CHECKPOINT 2

Tema Material 3 (claro e escuro)

Pasta: `checkpoints/02-tema`

- `main.dart` — ALTERADO
- `theme/app_theme.dart` — NOVO
- `theme/color_scheme.dart` — NOVO

- Gerar as cores no **Material Theme Builder**
- `ColorScheme` guarda papéis de cor
- `themeMode: ThemeMode.system` segue o aparelho



Modo claro **Modo escuro**

CHECKPOINT 2

Código: aplicando o tema

```dart
// main.dart
return MaterialApp(
  title: 'Minhas tarefas',
  debugShowCheckedModeBanner: false,
  theme: AppTheme.light,
  darkTheme: AppTheme.dark,
  themeMode: ThemeMode.system,
  home: const Scaffold(
    body: Center(child: Text('Olá, SAICC!')),
  ),
);
```

```dart
// theme/app_theme.dart
abstract final class AppTheme {
  static ThemeData get light =>
      _build(lightColorScheme);
  static ThemeData get dark =>
      _build(darkColorScheme);

  static ThemeData _build(ColorScheme scheme) {
    return ThemeData(
      useMaterial3: true,
      colorScheme: scheme,
      scaffoldBackgroundColor: scheme.surface,
      // appBarTheme, cardTheme, ...
    );
  }
}
```

`lightColorScheme` e `darkColorScheme` vêm do Theme Builder, em `theme/color_scheme.dart`. Nos widgets, use sempre `Theme.of(context).colorScheme`.

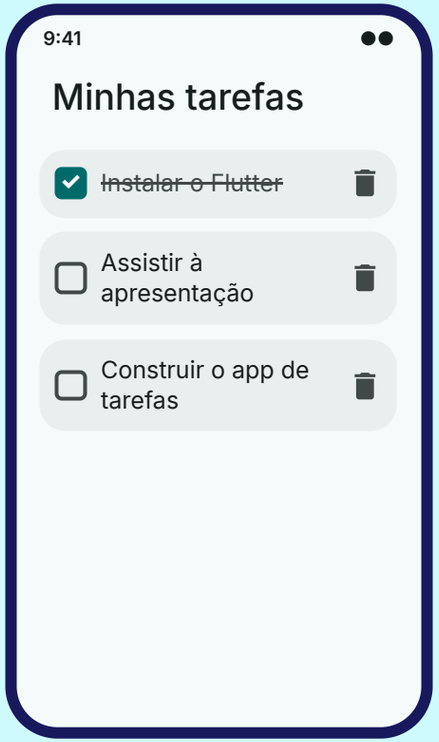
CHECKPOINT 3

Modelo e lista fixa

Pasta: `checkpoints/03-lista-fixa`

`main.dart`	ALTERADO
`models/todo.dart`	NOVO
`widgets/todo_tile.dart`	NOVO
`screens/home_screen.dart`	NOVO

- Classe `Todo` imutável, com `copyWith`
- `TodoTile` sem estado: dados descem, eventos sobem
- `ListView.separated` com dados de exemplo



Esperado: três cartões, o primeiro concluído

CHECKPOINT 3

Código: o modelo `Todo`

```dart
class Todo {
  const Todo({
    required this.id,
    required this.title,
    this.isDone = false,
  });

  factory Todo.create(String title) {
    return Todo(
      id: DateTime.now()
          .microsecondsSinceEpoch
          .toString(),
      title: title,
    );
  }
  // ...
```

```dart
  // ... continuação da classe
  final String id;
  final String title;
  final bool isDone;

  Todo copyWith({String? title, bool? isDone}) {
    return Todo(
      id: id,
      title: title ?? this.title,
      isDone: isDone ?? this.isDone,
    );
  }
}
```

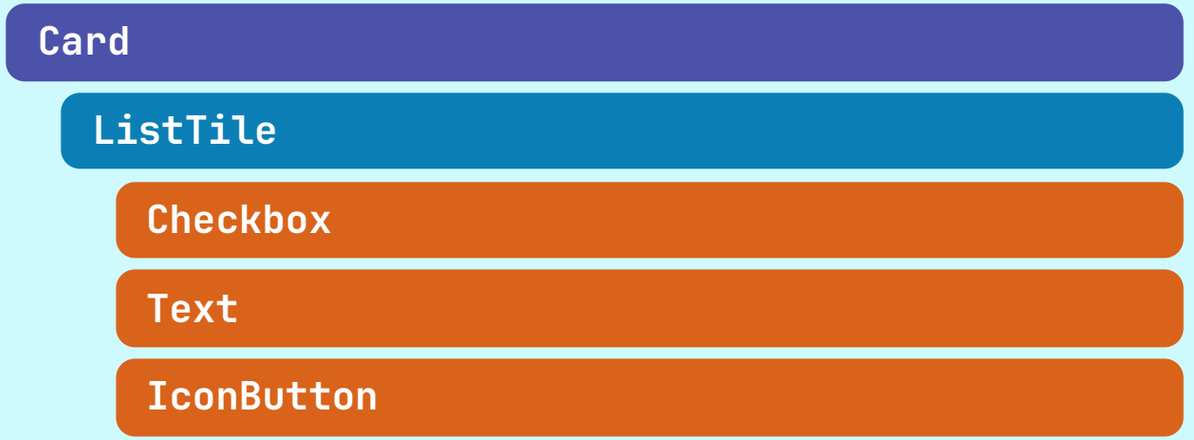
- Campos `final` e construtor `const`: a classe é **imutável**; `required` nos parâmetros nomeados
- `copyWith`: uma cópia alterada, sem mudar o original (`??` mantém o valor antigo quando o novo é nulo)

CHECKPOINT 3

Código: `TodoTile`

```dart
class TodoTile extends StatelessWidget {
  // ... campos final: todo, onToggle, onDelete

  @override
  Widget build(BuildContext context) {
    // ... textStyle: riscado se isDone
    return Card(
      child: ListTile(
        onTap: onToggle,
        leading: Checkbox(
          value: todo.isDone,
          onChanged: (_) => onToggle(),
        ),
        title: Text(todo.title, style: textStyle),
        trailing: IconButton(
          icon: const Icon(Icons.delete_outline),
          onPressed: onDelete,
        ),
      ),
    );
  }
}
```



- Composição: `Card` + `ListTile`
- Sem estado: só **callbacks** para avisar o pai
- Ternário no estilo: `isDone ? lineThrough : null`

CHECKPOINT 3

Código: `HomeScreen` com lista fixa

```dart
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  static const _todos = [
    Todo(id: '1', title: 'Instalar o Flutter',
        isDone: true),
    Todo(id: '2', title: 'Assistir à apresentação'),
    Todo(id: '3', title: 'Construir o app de tarefas'),
  ];
  // build ao lado
}
```

```dart
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Minhas tarefas')),
    body: SafeArea(
      child: ListView.separated(
        padding: const EdgeInsets.all(16),
        itemCount: _todos.length,
        separatorBuilder: (_, _) =>
            const SizedBox(height: 8),
        itemBuilder: (_, index) => TodoTile(
          todo: _todos[index],
          onToggle: () {},
          onDelete: () {},
        ),
      ),
    ),
  );
}
```

Os callbacks `() {}` ainda não fazem nada: vamos ligá-los no checkpoint 5.

CHECKPOINT 4

Adicionar tarefas com `setState`

Pasta: `checkpoints/04-adicionar-tarefas`

- `screens/home_screen.dart` **ALTERADO**
- `widgets/add_todo_sheet.dart` **NOVO**
- `widgets/empty_state.dart` **NOVO**

- `StatefulWidget` e `setState`
- Bottom sheet com `Form` e validação
- Estado vazio quando não há tarefas



1. Estado vazio | 2. Bottom sheet | 3. Tarefas na lista

CHECKPOINT 4

Código: o formulário (`AddTodoSheet`)

```dart
void _submit() {
  if (_formKey.currentState!.validate()) {
    Navigator.of(context).pop(_controller.text.trim());
  }
}

// dentro do Form
TextFormField(
  controller: _controller,
  validator: (value) {
    if (value == null || value.trim().isEmpty) {
      return 'Digite um título para a tarefa';
    }
    return null;
  },
),
```

Para comentar

- `showModalBottomSheet` devolve um `Future`: o título ou `null` se o usuário fechar
- `Form` + `validator`: validação sem `if` espalhado
- `dispose()` libera o `TextEditingController`
- Arquivo longo: pode copiar do checkpoint

CHECKPOINT 4

Código: a tela ganha estado

```dart
class _HomeScreenState extends State<HomeScreen> {
  List<Todo> _todos = [];

  Future<void> _addTodo() async {
    final title = await AddTodoSheet.show(context);
    if (title == null) return;

    setState(() => _todos = [Todo.create(title), ..._todos]);
  }

  Widget _buildBody() {
    if (_todos.isEmpty) {
      return const EmptyState();
    }
    return ListView.separated(
      padding: const EdgeInsets.fromLTRB(16, 8, 16, 96),
      itemCount: _todos.length,
      // ...
    );
  }
}
```

Para comentar

- `setState`: "o estado mudou, chame `build` de novo"
- Nova lista com spread (`...`), sem `list.add`
- Retorno antecipado: `if (vazio) return EmptyState()`
- Padding de 96 abaixo: o botão não cobre o último item

CHECKPOINT 5

Concluir, remover e desfazer

Pasta: `checkpoints/05-concluir-e-remover`

`screens/home_screen.dart` **ALTERADO**

- `_toggleTodo`: collection `for` com `if`
- `_deleteTodo` e `_restoreTodo`
- `SnackBar` com ação "Desfazer"
- `Key` única em cada item



1. Toque: conclui
2. Lixeira: remove e avisa
3. "Desfazer": volta no lugar

CHECKPOINT 5

Código: concluir uma tarefa

```dart
void _toggleTodo(Todo todo) {
  setState(() {
    _todos = [
      for (final item in _todos)
        if (item.id == todo.id)
          item.copyWith(isDone: !item.isDone)
        else
          item,
    ];
  });
}
// no itemBuilder
TodoTile(
  key: ValueKey(todo.id),
  todo: todo,
  onToggle: () => _toggleTodo(todo),
  onDelete: () => _deleteTodo(todo),
)
```

Para comentar

- `for` dentro da lista + `if/else`: faz o papel de um `map`
- `copyWith(isDone: !item.isDone)` troca só o campo
- `Key`: o Flutter sabe **qual** item é qual
- Ligamos os callbacks que estavam vazios

CHECKPOINT 5

Código: remover e desfazer

```dart
void _deleteTodo(Todo todo) {
  final index = _todos.indexWhere((t) => t.id == todo.id);
  if (index == -1) return;
  setState(() => _todos = [..._todos]..removeAt(index));

  ScaffoldMessenger.of(context)
    ..clearSnackBars()
    ..showSnackBar(
      SnackBar(
        content: Text('"${todo.title}" removida'),
        action: SnackBarAction(
          label: 'Desfazer',
          onPressed: () => _restoreTodo(todo, index),
        ),
      ),
    );
}

void _restoreTodo(Todo todo, int index) {
  setState(() {
    _todos = [..._todos]..insert(index.clamp(0, _todos.length), todo);
  });
}
```

Para comentar

- Guardamos o `index` para devolver o item ao mesmo lugar
- Cascata `..`: vários usos do mesmo objeto
- Interpolação `"${todo.title}"` no texto
- `SnackBarAction` chama `_restoreTodo`

CHECKPOINT 6 · SE DER TEMPO

Persistência com `shared_preferences`

Pasta: `checkpoints/06-persistencia`

- `data/todo_storage.dart` — NOVO
- `models/todo.dart` — ALTERADO
- `screens/home_screen.dart` — ALTERADO

- `flutter pub add shared_preferences`
- JSON no modelo: `toJson` e `fromJson`
- `async` / `await` no `initState`
- Pare e rode o app de novo (`flutter run`)



1. App com tarefas
2. Carregando do disco
3. As tarefas continuam lá

CHECKPOINT 6

Código: `TodoStorage`

```dart
class TodoStorage {
  static const _key = 'todos';
  Future<List<Todo>> load() async {
    final prefs = await SharedPreferences.getInstance();
    final raw = prefs.getString(_key);
    if (raw == null) return [];
    final items = jsonDecode(raw) as List<dynamic>;
    return items
        .map((i) => Todo.fromJson(i as Map<String, dynamic>))
        .toList();
  }
  Future<void> save(List<Todo> todos) async {
    final prefs = await SharedPreferences.getInstance();
    final raw = jsonEncode(todos.map((t) => t.toJson()).toList());
    await prefs.setString(_key, raw);
  }
}
```

Para comentar

- Disco é **assíncrono**: `Future`, `async` e `await`
- A lista vira texto JSON e volta com `jsonEncode` e `jsonDecode`
- Separar o armazenamento da tela facilita trocar por SQLite ou API

CHECKPOINT 6

Código: carregar e salvar na tela

```dart
final _storage = TodoStorage();
List<Todo> _todos = [];
bool _isLoading = true;
@override
void initState() {
  super.initState();
  _loadTodos();
}
Future<void> _loadTodos() async {
  final todos = await _storage.load();
  if (!mounted) return;
  setState(() {
    _todos = todos;
    _isLoading = false;
  });
}
void _saveTodos() => unawaited(_storage.save(_todos));
// em _buildBody: if (_isLoading) return spinner
```

Para comentar

- `_saveTodos()` depois de cada `setState` que muda a lista
- `mounted`: não chamar `setState` em tela destruída
- Spinner enquanto carrega: estado `_isLoading`
- Plugin novo exige parar e rodar de novo o app

FECHAMENTO

O que aprendemos em cada checkpoint

Checkpoint 1	`runApp`, `MaterialApp`, `Scaffold`, hot reload
Checkpoint 2	`ColorScheme`, Material 3, claro e escuro
Checkpoint 3	Classe imutável, widget sem estado, `ListView`
Checkpoint 4	`StatefulWidget`, `setState`, `Form`
Checkpoint 5	Callbacks, `Key`, `SnackBar`, desfazer
Checkpoint 6	`Future`, `async` / `await`, JSON, `mounted`

DESAFIOS

Para depois do curso

- Editar o título de uma tarefa (toque longo)
- Deslizar para remover com `Dismissible`
- Filtro: todas, pendentes e concluídas
- Botão "limpar concluídas" no `AppBar`

Depois disso

- Layout responsivo com `LayoutBuilder`
- Estado compartilhado: Provider, Riverpod, Bloc
- Publicar a versão web

OBRIGADO!

Perguntas?

Obrigado pela sua presença!



linkedin.com/in/matheuspsilveira